Hardware and software features to accelerate deep learning

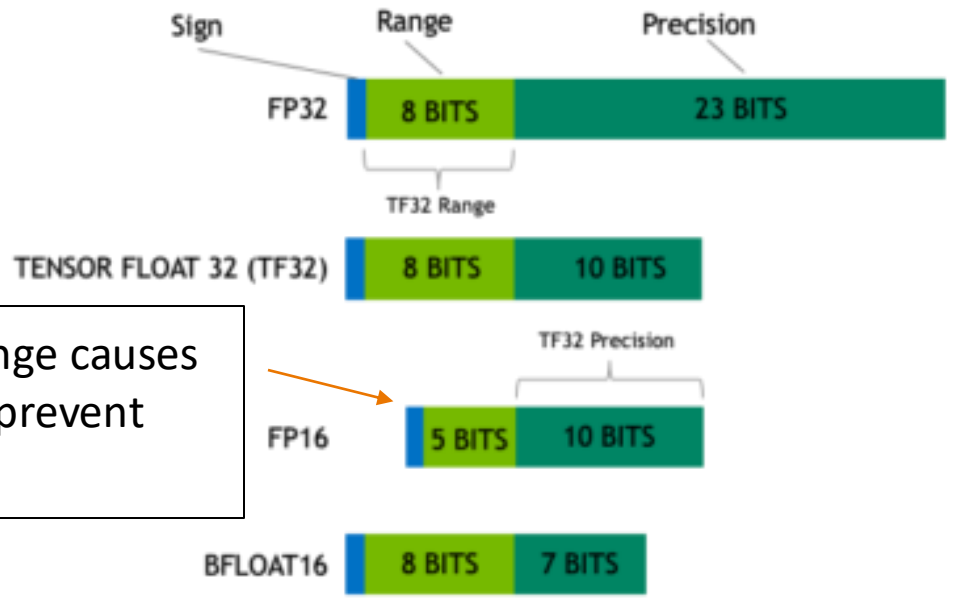
The three S'

- ~~Know your System~~
  - ~~What kind of hardware do I have?~~
  - ~~What features does my hardware support?~~
- Know your Software
  - Does my software do what I want to do efficiently?
  - Does my software support my hardware features?
- Know your Stack
  - What version of software am I running?
  - Is my stack compiled for my hardware?

Nvidia Ampere GPUs (e.g. A100, RTX 3090)

Features (see also https://servicedesk.surf.nl/wiki/display/WIKI/Deep+Learning+on+A100+GPUs)

- Tensor Cores support more datatypes (FP64, TF32, FP16, BF16, Int8, Int4, Binary)
- TensorFloat32 datatype:

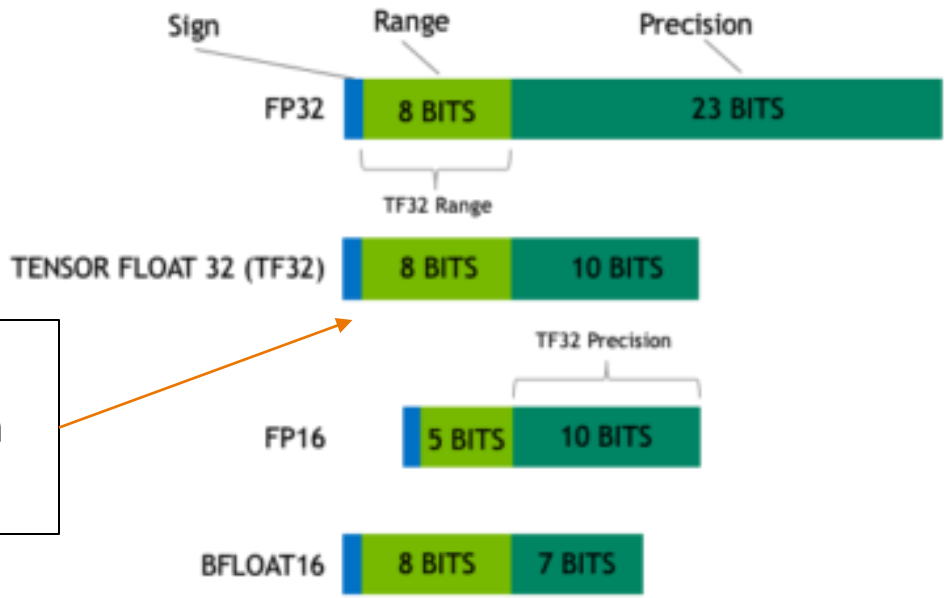


Mixed precision uses FP16. Reduced range causes (potential) need for gradient scaling to prevent underflows

Nvidia Ampere GPUs (e.g. A100, RTX 3090)

Features (see also https://servicedesk.surf.nl/wiki/display/WIKI/Deep+Learning+on+A100+GPUs)

- Tensor Cores support more datatypes (FP64, TF32, FP16, BF16, Int8, Int4, Binary)
- TensorFloat32 datatype:

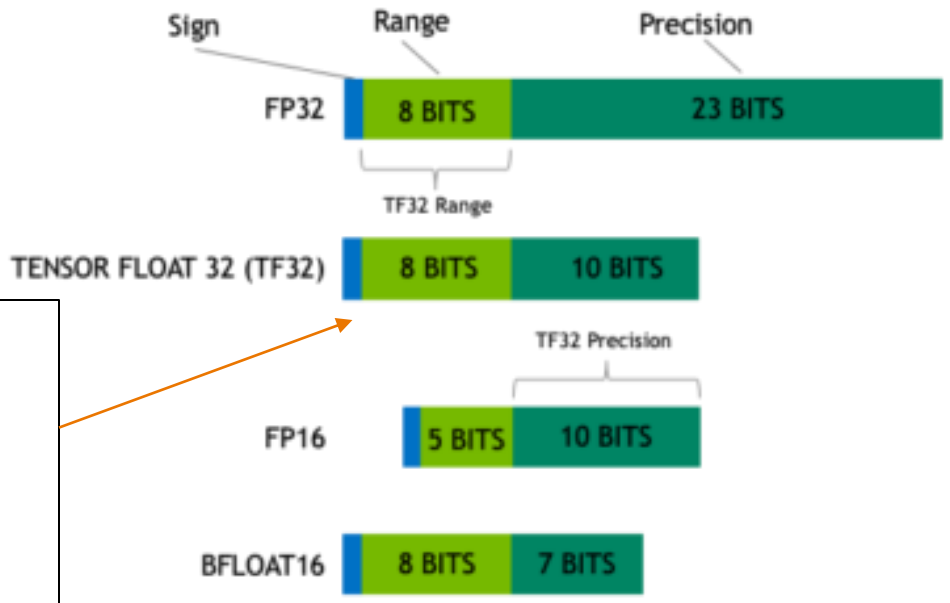


TF32 tries to combine best of both worlds: range of an FP32 (no gradient scaling needed), but with reduced precision (for speedup & less memory)

Nvidia Ampere GPUs (e.g. A100, RTX 3090)

Features (see also https://servicedesk.surf.nl/wiki/display/WIKI/Deep+Learning+on+A100+GPUs)

- Tensor Cores support more datatypes (FP64, TF32, FP16, BF16, Int8, Int4, Binary)
- TensorFloat32 datatype:



Automatically used *unless*
- Environment variable *NVIDIA_TF32_OVERRIDE=0* is set
- *torch.backends.cuda.matmul.allow_tf32 = False* and torch.*backends.cudnn.allow_tf32 = False* are set

Second S: Software

- PyTorch/TensorFlow vs Numpy
  - PyTorch and TensorFlow have GPU support, Numpy does not
- Support for CUDA (Nvidia), ROCm (AMD), <insert exotic hardware here>?
  - E.g.: Intel SapphireRapids
  - IPEX: https://github.com/intel/intel-extension-for-pytorch
  - LUMI https://www.lumi-supercomputer.eu/
- Support for lower precision
  - BFloat16, Float16? FP8?

Third S: Stack

- Is my software compiled for my hardware
  - Pip install … ?
  - Module load PyTorch/1.12.0-foss-2022a-CUDA-11.7.0
  - https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/30660245/Environment+Modules
- Again: Consider the hardware/platform that you're using
  - GPU: Nvidia/CUDA? AMD/ROCm?
  - CPUs: AVX512?

Wait ... so what happens?

- You call torch
  - E.g. scatter_add_ -> Message passing in Graph-NNs
  - (Manual) Dispatching
- Torch (python) determines the most appropriate function to call
  - Dispatching
- Torch (C++/CU) calls the CPU/GPU kernel
  - Dispatching
- Low-level C++/CU library performs actual calculation
  - Dispatching
- PyTorch Dispatcher:

http://blog.ezyang.com/2020/09/lets-talk-about-the-pytorch-dispatcher/

In practice: PyTorch + Snellius

- We only have NVIDIA GPUs
  - 72x4=288 NVIDIA A100
  - 88x4=352 NVIDIA H100
- Makes optimization a bit easier for you
- You only need to spend a year sifting through documentation
  - https://servicedesk.surf.nl/wiki/display/WIKI/Deep+Learning+on+A100+GPUs
  - Or ask us... :)
  - Every EINF/NWO small grants has a default of 4 consultancy hours

Rapidfire: implementation details

- How fast do you want to go?
- What is your goal?
  - Largest model possible?
  - Best validation performance?
  - Submit a paper to ISC ‘25?

Table 1. Examples of neural network operations with their arithmetic intensities. Limiters assume FP16 data and an NVIDIA V100 GPU.

Operation	Arithmetic Intensity	Usually limited by...
Linear layer (4096 outputs, 1024 inputs, batch size 512)	315 FLOPS/B	arithmetic
Linear layer (4096 outputs, 1024 inputs, batch size 1)	1 FLOPS/B	memory
Max pooling with 3x3 window and unit stride	2.25 FLOPS/B	memory
ReLU activation	0.25 FLOPS/B	memory
Layer normalization	< 10 FLOPS/B	memory

Rapidfire: implementation details

- Some useful resources:
  - https://pytorch.org/docs/stable/notes/cuda.html
  - https://docs.nvidia.com/deeplearning/performance/dl-performance-memory-limited/index.html
  - https://docs.nvidia.com/deeplearning/performance/index.html
- Duncan: *"You can take any piece of code, spend 6 months optimizing it, and then write a paper about it"*

Level 1: Shouldn't impact model convergence:

- Avoid CPU-GPU sync

Create tensors directly on the device

Also applicable to:

`torch.empty()`, `torch.zeros()`, `torch.full()`, `torch.ones()`, `torch.eye()`, `torch.randint()`, `torch.randn()`

and similar functions.

Level 1: Shouldn't impact model convergence:

- Avoid doing these inside a training loop if possible:
  - Operations which require synchronization:
    - `print(cuda_tensor)`
    - `cuda_tensor.item()`
    - memory copies: `tensor.cuda()`, `cuda_tensor.cpu()` and `tensor.to(device)` calls
    - `cuda_tensor.nonzero()`
    - python control flow which depends on operations on CUDA tensors e.g.

```
if (cuda_tensor != 0).all()
```

This forces PyTorch to copy data from GPU to CPU, slowing performance.

Level 1: Shouldn't impact model convergence:

- Don't aggregate tensors which still require grad, e.g.: losses.append(loss) (:o)
  - instead, detach (loss.detach())

Each loss in losses.append(loss) stores the entire computation graph. PyTorch stores extra gradients unnecessarily!

```
losses = []  # List to store loss values
for data, target in dataloader:
    output = model(data)
    loss = criterion(output, target)  # Compute loss

    losses.append(loss)  # ❌ BAD: Keeps computation graph alive!

    losses.append(loss.detach())  # ✅ Detaches tensor, prevents memory issues
```

detach() creates a new tensor with the same values but removes the gradient tracking.

Level 1: Shouldn't impact model convergence:

- torch.backends.cuda.matmul.allow_tf32 = True
  - The default in PyTorch (silently)
- torch.set_float32_matmul_precision('high')
- torch.backends.cudnn.benchmark = True
  - Empirically benchmarks to find fastest algorithm for some methods, default in PyTorch
- Align bytes: multiples of 2, 8, 1024
  - DNN libraries want nicely aligned memory
- torch.channels_last
  - Helps with memory alignment and reduces cache misses
- Compile: PyTorch 2.0
  - Big speedup, but sensitive to implementation

There is little reason *not* to use TensorFloat32 ☺

Helps maintain accuracy in cases where TF32 might introduce minor errors

Stores images in NHWC format instead of NCHW (optimizes GPU cache usage)

```
eager train time 0: 0.26062847900390623
eager train time 1: 0.05141097640991211
eager train time 2: 0.04929536056518555
eager train time 3: 0.048909313201904295
```

```
compile train time 0: 222.846328125
compile train time 1: 9.1764130859375
compile train time 2: 0.025482240676879882
compile train time 3: 0.022506496429443360
```

```
(train) eager median: 0.04906649589538574, compile median: 0.02168064022064209,
speedup: 2.263147923494881x
```

Level 2: Danger zone

Lower precision (FP16 or BF16) for matrix reductions instead of higher precision (FP32, advised). Can cause numerical instability!

- torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction = True
- Lol: A similar flag (as above) exists for BFloat16 GEMMs. Note that this switch is set to *False* by default for BF16 as we have observed numerical instability in PyTorch CI tests (e.g., test/test_matmul_cuda.py).
- torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction = True

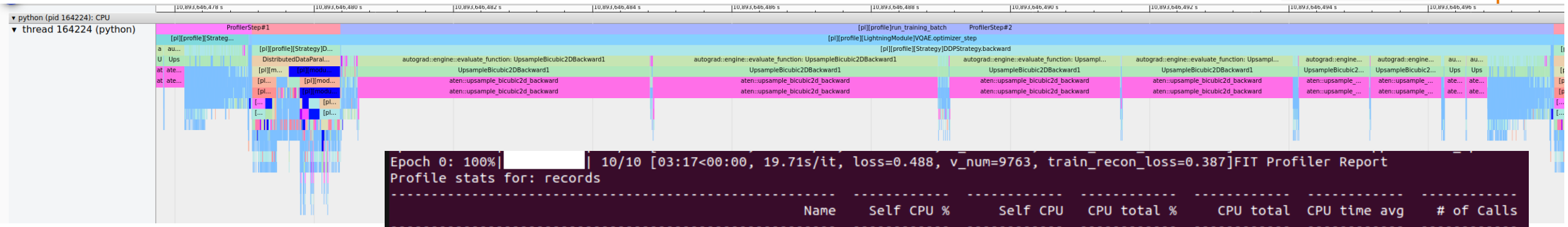
trades some accuracy for faster compute. Can affect model convergence

- Torch.set_float32_matmul_precision('medium')
- Cast model and data to (B)Float16 before the forward pass
  - We're not supposed to...but:

Forces all computations to use lower precision, instead of letting autocast() to choose

Change your model

- Worst case scenario:




```
Epoch 0: 100%|          | 10/10 [03:17<00:00, 19.71s/it, loss=0.488, v_num=9763, train_recon_loss=0.387]FIT Profiler Report
Profile stats for: records
---------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------
                                                     Name    Self CPU %      Self CPU   CPU total %     CPU total  CPU time avg    # of Calls
---------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------
                                                ProfilerStep*         2.77%       2.611s        60.21%       56.680s       18.893s             3
                      [pl][profile][Strategy]DDPStrategy.backward         0.11%     106.540ms        55.55%       52.298s       17.433s             3
autograd::engine::evaluate_function: UpsampleBicubic...                  0.00%     410.000us        50.59%       47.628s        1.985s            24
                                    UpsampleBicubic2DBackward1            0.00%     153.000us        50.59%       47.628s        1.984s            24
                                 aten::upsample_bicubic2d_backward       50.33%      47.382s        50.59%       47.628s        1.984s            24
                                   [pl][profile]run_training_batch         0.00%       3.242ms        41.60%       39.158s       13.053s             3
                [pl][profile][LightningModule]VQAE.optimizer_step        37.11%      34.931s        41.59%       39.155s       13.052s             3
                 [pl][profile][Strategy]DDPStrategy.training_step         0.00%     346.000us         4.49%        4.224s        1.408s             3
                                    DistributedDataParallel.forward         0.01%       9.523ms         4.49%        4.223s        1.408s             3
                          [pl][module]vq_ae.model.Decoder: decoder          0.00%     540.000us         2.58%        2.427s      808.864ms             3
autograd::engine::evaluate_function: torch::autograd...                  0.02%      19.580ms         2.54%        2.391s        1.779ms          1344
torch::autograd::CppNode<torch_ipex::cpu::IPEXConvol...                 -0.00%    -448.000us         2.52%        2.372s        1.765ms          1344
                                       IPEXConvolutionOp::backward       0.01%       9.489ms         2.52%        2.369s        1.763ms          1344
                                  torch_ipex::convolution_backward       2.43%       2.292s         2.51%        2.363s        1.758ms          1344
                          [pl][module]vq_ae.model.Encoder: encoder          0.00%     732.000us         1.89%        1.779s      592.886ms             3
[pl][module]vq_ae.layers.conv_block.UpBlock: decoder...                  0.00%      84.000us         1.82%        1.715s      571.554ms             3
[pl][module]torch.nn.modules.container.Sequential: d...                  0.00%     416.000us         1.82%        1.715s      571.525ms             3
                                                        aten::add        1.56%       1.464s         1.56%        1.464s      359.683us          4071
                                  torch_ipex::convolution_forward       -0.01%    -8343.000us         1.44%        1.357s        1.010ms          1344
                                       IPEXConvolutionOp::forward       0.03%      31.301ms         1.42%        1.338s      995.645us          1344
---------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------
Self CPU time total: 94.141s
```

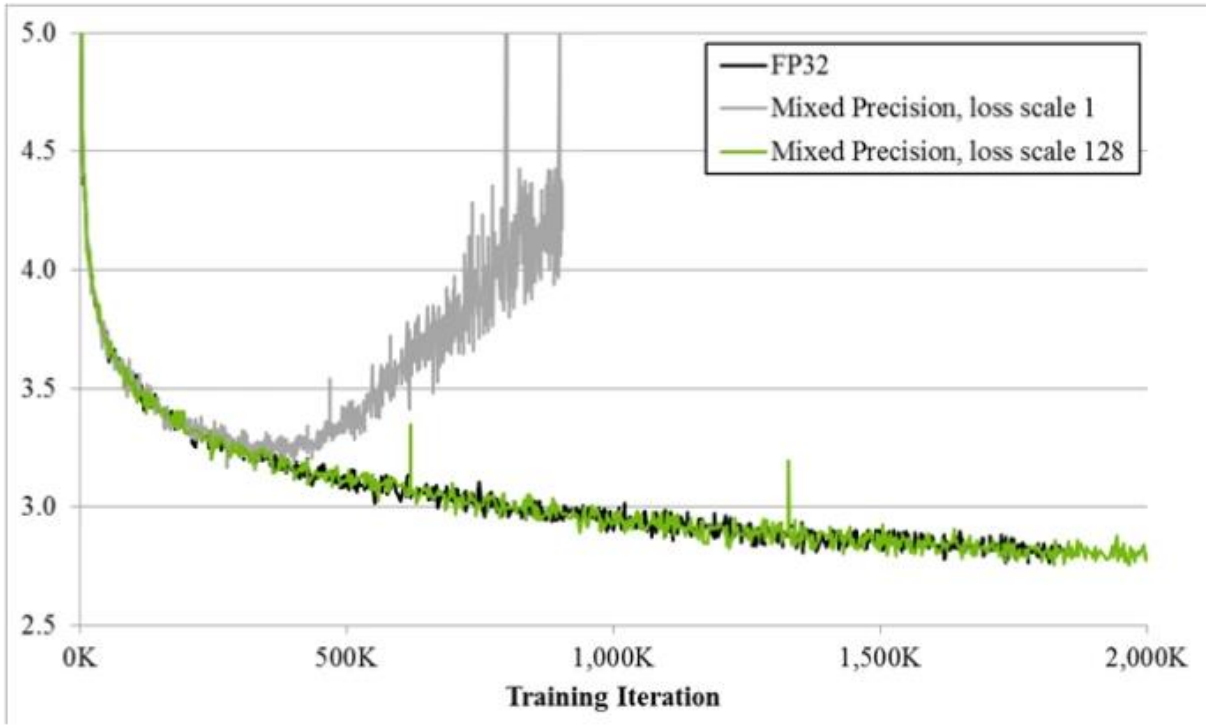
Change your model

- Change model architecture to improve performance characteristics, e.g.:
  - Use LoRA for fine-tuning
  - Use pre-trained model aka zero shot (@train VAE for latent space creation or use pre-trained model)
    - E.g. DINOv2 (not DINo from yesterday)
  - Incorporate dataloading operations into the model instead of CPU pipeline
  - ViT vs CNN vs GNN
  - Change non-linearities
  - Multiples of 2,8,1024
    - Channels
    - Hidden size
    - Vocab size
    - Batch size
    - Input resolution/sequence length
    - Everything
  - For transformers:
    - Patching
    - Masked attention

Training Time	CNN Models					
Activations	MobileNet	VGG16	GoogleNet	ResNet50	SENet18	DenseNet121
Sigmoid	00:33:15	00:49:16	04:55:54	03:36:03	01:13:14	04:12:24
Tanh	00:33:18	00:49:55	04:58:02	03:33:03	01:13:18	04:09:24
Elliott [25]	00:49:52	00:59:13	06:53:55	05:38:49	01:41:38	07:46:55
ReLU [8]	00:31:22	00:47:19	04:55:10	03:32:30	01:15:33	04:15:06
LReLU [34]	00:31:48	00:49:03	05:01:30	03:33:00	01:18:38	04:14:09
PReLU [35]	00:44:24	00:49:01	05:42:18	03:55:57	01:27:05	04:55:47
ELU [27]	00:31:05	00:47:38	04:57:37	03:36:47	01:13:25	04:08:39
SELU [52]	00:29:40	00:47:31	04:54:57	03:33:47	01:13:27	04:09:17
GELU [101]	00:29:43	00:47:22	04:55:53	03:32:32	01:13:32	04:11:26
CELU [53]	00:29:36	00:46:47	05:00:44	03:31:40	01:14:08	04:18:11
Softplus [93]	00:29:44	00:47:06	04:58:55	03:32:03	01:14:02	04:12:08
Swish [29]	00:43:13	00:55:37	06:18:38	04:58:38	01:32:15	06:41:14
ABReLU [44]	00:38:51	00:53:49	05:43:59	04:27:02	01:25:30	05:42:53
LiSHT [24]	00:37:01	00:54:10	05:40:00	04:25:57	01:23:59	05:38:15
SRS [26]	01:06:38	01:11:36	08:43:09	07:35:35	02:05:33	11:10:27
Mish [99]	00:40:19	00:54:23	05:59:48	04:46:45	01:28:53	06:10:27
PAU [111]	00:41:59	00:54:10	05:54:22	04:12:31	01:25:37	05:39:57
PDELU [59]	05:23:38	04:01:55	34:22:00	36:48:48	08:32:40	50:23:00

Always

- Have a baseline
- Check convergence statistics



Profile

Get a grip on your model’s performance

- Profile
- Profile
- Profile